



INSTITUTO POLITÉCNICO NACIONAL
ESCUELA SUPERIOR DE CÓMPUTO



TERAPHYBENEFIT

Reporte Final de Implementación, Integración y Validación



Unidad de aprendizaje: Análisis y Diseño de Software

Grupo: 5CM4

Equipo: Aparicio Arenas Victor Eduardo

García Alvarado Diego

Guerrero Zamora Pablo Alexis

Licon Morales Diego

Zuluaga Pérez Zianya Victoria

Fecha de entrega: 22/06/2026

Carátula de versiones

Versión	Fecha	Autor(es)	Descripción del cambio
1.0	27/05/2026	Equipo 5CM4	Inicio de planeación del proyecto y revisión de artefactos solicitados.
2.0	29/05/2026	García Alvarado Diego	Definición y refinamiento de requerimientos, modelo de análisis y lineamientos de modelado.
3.0	03/06/2026	García Alvarado Diego	Actualización del modelo de diseño, arquitectura y estructura lógica del sistema.
4.0	09/06/2026	Aparicio Arenas Victor Eduardo y Guerrero Zamora Pablo Alexis	Desarrollo de prototipo visual para validar distribución de interfaz y flujo principal.
5.0	15/06/2026	Licona Morales Diego y Zuluaga Pérez Zianya Victoria	Implementación final en PHP/MySQL, integración de módulos y correcciones visuales.
6.0	17/06/2026	Equipo 5CM4	Reporte final con evidencia, pruebas, asignación de responsabilidades y entregables.

Índice

Carátula de versiones	2
1. Resumen ejecutivo	5
2. Antecedentes y justificación del proyecto.....	5
3. Objetivos.....	5
3.1 Objetivo general	5
3.2 Objetivos específicos	5
4. Equipo de trabajo y asignación de responsabilidades.....	6
5. Cronograma y control del proyecto.....	6
6. Alcance final de la versión implementada.....	7
7. Requerimientos implementados	7
7.1 Requisitos no funcionales	7
8. Relación con artefactos de análisis y diseño	8
9. Modelo de análisis aplicado	8
10. Modelo de diseño y arquitectura	10
11. Modelo de datos final.....	11
12. Implementación del sistema	12
13. Funcionalidades por rol	12
13.1 Administrador.....	12
13.2 Doctor	12
13.3 Paciente	13
14. Evidencia visual de implementación.....	13
15. Prototipo visual y evolución hacia el sistema funcional	17
17. Manual de usuario resumido.....	18
17.1 Acceso al sistema.....	18
17.2 Flujo del administrador.....	18
17.3 Flujo del doctor	18
17.4 Flujo del paciente	19
18. Manual técnico resumido	19
19. Seguridad y control de acceso	19
20. Resultados obtenidos	19
21. Conclusiones	20
22. Trabajo futuro.....	20

23. Anexos21

23.1 Paquete entregable21

23.2 Evidencias adicionales21

1. Resumen ejecutivo

TeraphyBenefit es una aplicación web para la gestión clínica de pacientes en rehabilitación física. El sistema fue desarrollado con una estructura basada en PHP, MySQL, HTML, CSS y JavaScript, ejecutándose localmente mediante XAMPP. La versión final integra autenticación por roles, administración de doctores, gestión de pacientes, agenda de citas, registro de notas SOAP, portal de paciente, tienda de productos terapéuticos, carrito de compras, pedidos y reportes clínicos.

El objetivo del presente reporte es documentar la implementación final del producto, relacionándola con los artefactos de requerimientos, análisis y diseño previamente elaborados. También se incorporan evidencias del proceso de desarrollo, pruebas, base de datos, módulos terminados y distribución de responsabilidades del equipo.

Resultado principal

La entrega final transforma el prototipo y los modelos documentales en un producto funcional, con base de datos relacional, control de acceso por roles y módulos operativos para administrador, doctor y paciente.

2. Antecedentes y justificación del proyecto

En los documentos de requerimientos se identificó que muchos consultorios de rehabilitación física continúan utilizando registros manuales, hojas de cálculo o formatos aislados para administrar pacientes. Este método dificulta el seguimiento continuo de tratamientos, aumenta el riesgo de pérdida de información y genera carga administrativa para el personal clínico.

TeraphyBenefit surge como una propuesta orientada a centralizar el expediente del paciente, documentar sesiones, programar citas y visualizar la evolución de cada tratamiento. La versión final amplía el alcance inicial al incluir un portal para el paciente y una tienda de artículos de apoyo para ejercicios terapéuticos, lo cual permite que el seguimiento clínico se complemente con recomendaciones de productos de rehabilitación.

3. Objetivos

3.1 Objetivo general

Desarrollar e integrar una aplicación web funcional para la administración de pacientes en rehabilitación física, incorporando módulos clínicos, administrativos y de seguimiento que permitan mejorar la organización de la información y apoyar la continuidad del tratamiento.

3.2 Objetivos específicos

- Implementar autenticación de usuarios con roles diferenciados: administrador, doctor y paciente.
- Permitir que el administrador registre y elimine doctores, además de gestionar productos de la tienda.
- Permitir que el doctor registre pacientes, agende citas, capture notas SOAP y consulte reportes.
- Permitir que el paciente consulte su historial de sesiones, citas y recomendaciones clínicas.
- Integrar una tienda con productos terapéuticos, carrito de compras y registro de pedidos.
- Diseñar una base de datos relacional que garantice persistencia y trazabilidad de la información.
- Documentar evidencias de implementación, pruebas y resultados finales.

4. Equipo de trabajo y asignación de responsabilidades

El equipo pertenece al grupo 5CM4 y estuvo conformado por cinco integrantes. La asignación se documenta para dejar evidencia de la distribución de actividades y de las contribuciones realizadas durante el desarrollo.

Integrante	Responsabilidad principal	Actividades realizadas	Entregable asociado
Aparicio Arenas Victor Eduardo	Prototipo visual	Participó en la construcción de un prototipo inicial visual para validar la idea general de la interfaz, distribución de pantallas y navegación básica.	Prototipo de interfaz visual.
García Alvarado Diego	Modelos y requerimientos	Elaboró y refinó la parte de requerimientos, modelos de análisis/diseño y apoyo en la definición de reglas, actores y flujo del sistema.	SRS, modelo de análisis y modelo de diseño.
Guerrero Zamora Pablo Alexis	Prototipo visual	Apoyó en la elaboración del prototipo visual inicial y en la revisión de la propuesta de interfaz antes de la implementación funcional.	Prototipo de interfaz visual.
Licona Morales Diego	Implementación del código	Desarrolló la versión funcional en PHP/MySQL, integrando autenticación, pacientes, citas, notas, reportes, tienda y ajustes de interfaz.	Código final ejecutable.
Zuluaga Pérez Zianya Victoria	Implementación del código	Participó en la construcción e integración del código final, correcciones visuales, pruebas de ejecución y validación del flujo funcional.	Código final ejecutable y pruebas.

Nota de colaboración

Licona Morales Diego y Zuluaga Pérez Zianya Victoria realizaron la implementación funcional del código. García Alvarado Diego trabajó principalmente la parte de modelos y requerimientos. Aparicio Arenas Victor Eduardo y Guerrero Zamora Pablo Alexis desarrollaron un prototipo visual que sirvió como referencia de diseño.

5. Cronograma y control del proyecto

El cronograma se organizó considerando como primer día el 27/05/2026 y como último día el 17/06/2026. El periodo fue dividido por etapas: planeación, requerimientos, análisis, diseño, implementación, pruebas y entrega final.

Cronograma del proyecto (27/05/2026 - 17/06/2026)



Figura 1. Cronograma general del proyecto.

Fecha	Actividad	Descripción	Entregable	Estado
27/05/2026	Inicio y planeación	Definición del problema, revisión de recursos solicitados y organización del equipo.	Planeación inicial	Completado
29/05/2026	Requerimientos	Elaboración de SRS, actores, stakeholders, reglas de negocio y backlog.	Requerimientos	Completado
01/06/2026	Modelo de análisis	Casos de uso, contexto, historias de usuario, criterios de aceptación y comportamiento.	Modelo de análisis	Completado
03/06/2026	Modelo de diseño	Arquitectura, clases, componentes y despliegue.	Modelo de diseño	Completado
09/06/2026	Implementación	Desarrollo de módulos PHP/MySQL y primeras pruebas locales.	Código funcional	Completado
15/06/2026	Pruebas y ajustes	Validación de login, roles, formularios, tienda, base de datos e interfaz.	Evidencia de pruebas	Completado
17/06/2026	Entrega final	Integración del reporte, código y evidencias de funcionamiento.	Entrega del proyecto	Completado

6. Alcance final de la versión implementada

La versión implementada supera el alcance inicial documentado en los primeros requerimientos, debido a que se agregaron funcionalidades de portal de paciente y tienda. En la versión final, el sistema no solo administra información clínica, sino que también permite al paciente consultar su seguimiento y adquirir artículos de apoyo para ejercicios de rehabilitación.

Módulo	Descripción funcional	Rol que lo utiliza
Autenticación	Inicio de sesión mediante usuario y contraseña, con redirección según rol.	Admin, doctor, paciente
Panel admin	Vista administrativa para revisar doctores, productos y pedidos.	Admin
Gestión de doctores	Alta y eliminación de doctores con contraseña segura y datos de contacto.	Admin
Gestión de pacientes	Registro, listado y eliminación de pacientes; creación opcional de usuario para portal.	Doctor
Agenda	Registro de citas con paciente, fecha, hora, motivo y notas.	Doctor
Notas SOAP	Captura de sesión terapéutica, signos vitales, ejercicios, evolución, dolor y recomendaciones.	Doctor
Portal de paciente	Consulta de expediente, sesiones, citas y seguimiento del tratamiento.	Paciente
Tienda	Visualización de productos terapéuticos con imágenes relacionadas.	Paciente
Carrito y pedidos	Selección de productos, compra y vaciado de carrito.	Paciente
Productos	Alta y baja de productos de tienda con imagen, precio, stock y descripción.	Admin
Reportes	Resumen de evolución clínica por sesiones y dolor promedio.	Doctor

7. Requerimientos implementados

Los requerimientos funcionales se consolidaron tomando como base el SRS, el modelo de análisis y los cambios realizados durante la implementación. La siguiente matriz muestra la trazabilidad entre requisitos y módulos finales.

ID	Requerimiento	Implementación final	Estado
RF-01	Registrar pacientes con datos básicos.	patients.php permite registrar nombre, edad, género, lesión, contacto, fecha, estado y nota inicial.	Implementado
RF-02	Registrar sesiones terapéuticas.	sessions.php captura notas SOAP con fecha, hora, terapia, ejercicios, signos vitales, evolución, dolor y recomendaciones.	Implementado
RF-03	Generar seguimiento del progreso.	reports.php resume sesiones y dolor promedio; patient_portal.php muestra historial y últimas métricas.	Implementado
RF-04	Consultar historial clínico.	El doctor consulta sesiones en Notas SOAP y el paciente consulta su historial en el portal.	Implementado
RF-05	Gestionar citas.	appointments.php registra citas por paciente con fecha, hora, motivo, estado y notas.	Implementado
RF-06	Buscar y filtrar expedientes.	La barra superior permite búsqueda rápida por paciente desde el panel médico.	Implementado
RF-07	Autenticación y roles.	login.php valida usuario/contraseña y helpers.php controla permisos de admin, doctor y paciente.	Implementado
RF-08	Gestionar doctores.	doctors.php permite alta y eliminación de doctores; solo disponible para admin.	Implementado
RF-09	Gestionar tienda.	products.php permite alta/baja de productos con precio, stock e imagen; solo admin.	Implementado
RF-10	Carrito de compras.	store.php y cart.php permiten agregar productos, revisar carrito y finalizar compra.	Implementado

7.1 Requisitos no funcionales

ID	Requisito no funcional	Evidencia de cumplimiento
RNF-01	Usabilidad	Diseño con navegación lateral, tarjetas, formularios ordenados y separación por roles.
RNF-02	Seguridad	Control de acceso por sesión y validación de roles; restricción de módulos por tipo de usuario.
RNF-03	Integridad de datos	Uso de claves primarias y foráneas en MySQL; eliminación en cascada/controlada.
RNF-04	Mantenibilidad	Separación de archivos por módulos PHP, configuración de base de datos y estilos CSS independientes.
RNF-05	Portabilidad local	Ejecución local en XAMPP mediante Apache, PHP y MySQL.

8. Relación con artefactos de análisis y diseño

El reporte final se construye a partir de los documentos previos del proyecto: Especificación de Requisitos de Software, Modelo de Análisis, Modelo de Diseño y documentación complementaria. Estos artefactos sirven como base para justificar la implementación final.

Artefacto	Contenido utilizado	Aplicación en el reporte final
SRS / Requerimientos	Visión del producto, actores, RF, RNF, reglas de negocio y backlog.	Trazabilidad de requisitos, alcance y justificación.
Modelo de Análisis	Contexto, casos de uso, historias de usuario, criterios de aceptación y comportamiento.	Validación de funcionalidades y flujos de usuario.
Modelo de Diseño	Arquitectura MVC, clases, robustez, componentes y despliegue.	Explicación técnica de la estructura del sistema.
Complemento extendido	Narrativa del equipo, plan de pruebas, manuales, cronograma y conclusiones.	Ampliación del reporte con evidencias y responsabilidades.
Código final	PHP, MySQL, CSS, JavaScript, assets e imágenes de productos.	Evidencia de implementación y funcionamiento.

9. Modelo de análisis aplicado

El modelo de análisis define los actores y casos de uso que explican el comportamiento esperado del sistema. Para la versión final, los actores se refinan de la siguiente manera: el administrador administra doctores y tienda; el doctor gestiona pacientes, citas, notas y reportes; el paciente consulta su seguimiento y realiza compras en la tienda.

Modelo de contexto - TeraphyBenefit

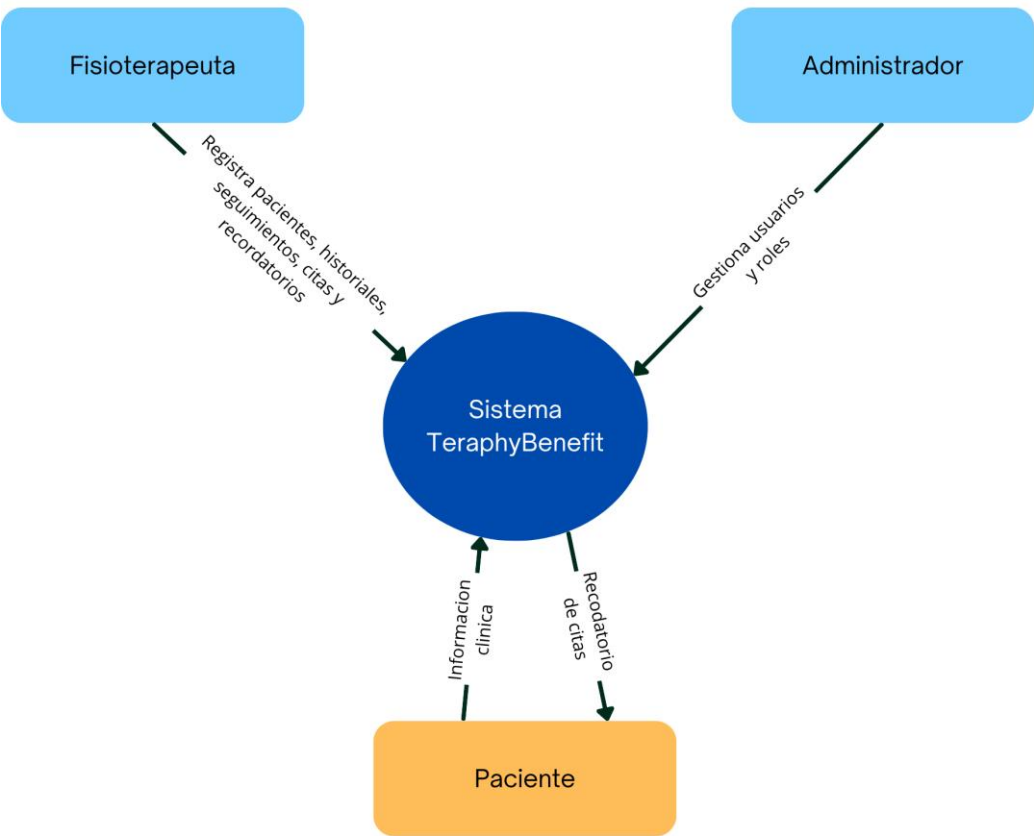


Figura 2. Modelo de contexto del sistema utilizado como base de análisis.

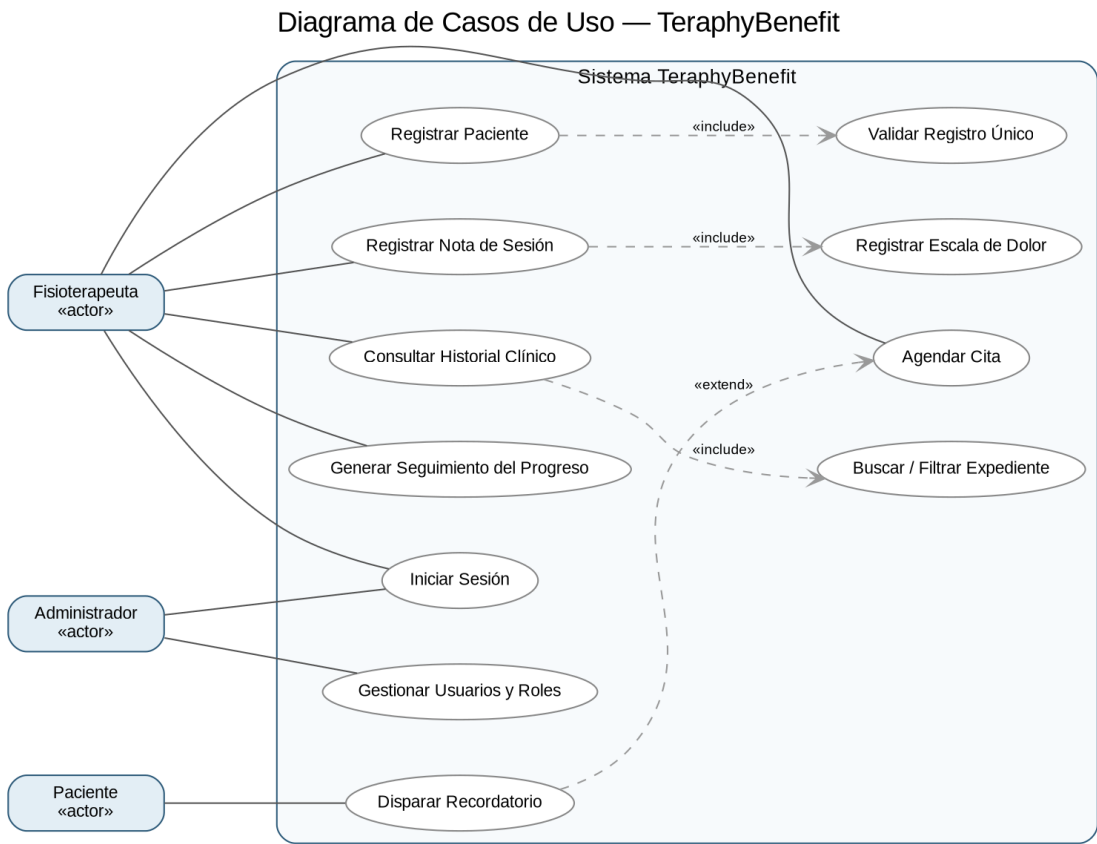


Figura 3. Diagrama de casos de uso base del sistema TeraphyBenefit.

Actor	Responsabilidades en la versión final
Administrador	Crea y elimina doctores; registra productos de tienda; consulta pedidos generales.
Doctor	Registra pacientes, agenda citas, captura sesiones SOAP, consulta reportes y da seguimiento clínico.
Paciente	Consulta su historial de sesiones y citas; visualiza recomendaciones; compra productos de rehabilitación.

10. Modelo de diseño y arquitectura

La implementación conserva una arquitectura cliente-servidor organizada en capas. El navegador funciona como cliente y consume vistas PHP renderizadas en HTML. La lógica de negocio se distribuye en archivos PHP por módulo, mientras que la persistencia se realiza en MySQL. El acceso a la base de datos se centraliza mediante el archivo config/database.php.

Estilo Arquitectónico — Cliente-Servidor en Capas (patrón MVC, aplicación web)

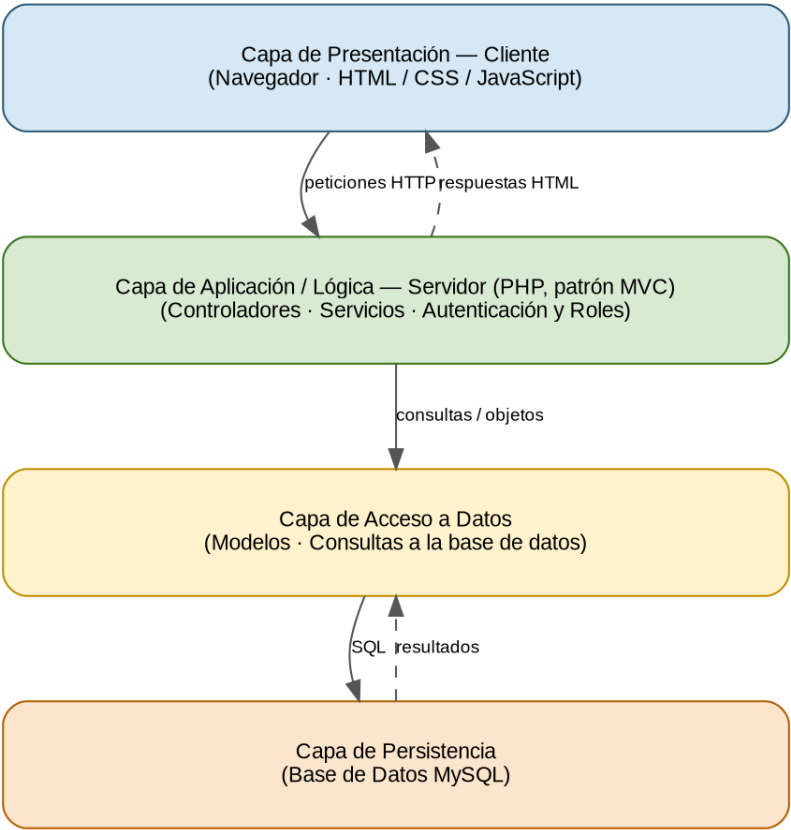


Figura 4. Estilo arquitectónico cliente-servidor en capas usado como referencia.

Capa	Componentes	Responsabilidad
Presentación	HTML, CSS, JavaScript, vistas PHP	Renderizar interfaz, formularios, tablas, tarjetas, tienda y portal.
Aplicación / lógica	login.php, helpers.php, módulos PHP	Validar sesión, controlar roles, procesar formularios y coordinar casos de uso.
Acceso a datos	config/database.php, consultas PDO	Conectar con MySQL, ejecutar consultas e insertar registros.
Persistencia	database/teraphybenefit.sql	Definir tablas, claves, restricciones y datos iniciales.

11. Modelo de datos final

La base de datos final se denomina teraphybenefit y contiene entidades para usuarios, doctores, pacientes, citas, sesiones, productos, carrito y pedidos. Se utilizaron claves primarias autoincrementales y relaciones mediante claves foráneas para mantener la integridad referencial.

Modelo relacional final - TeraphyBenefit

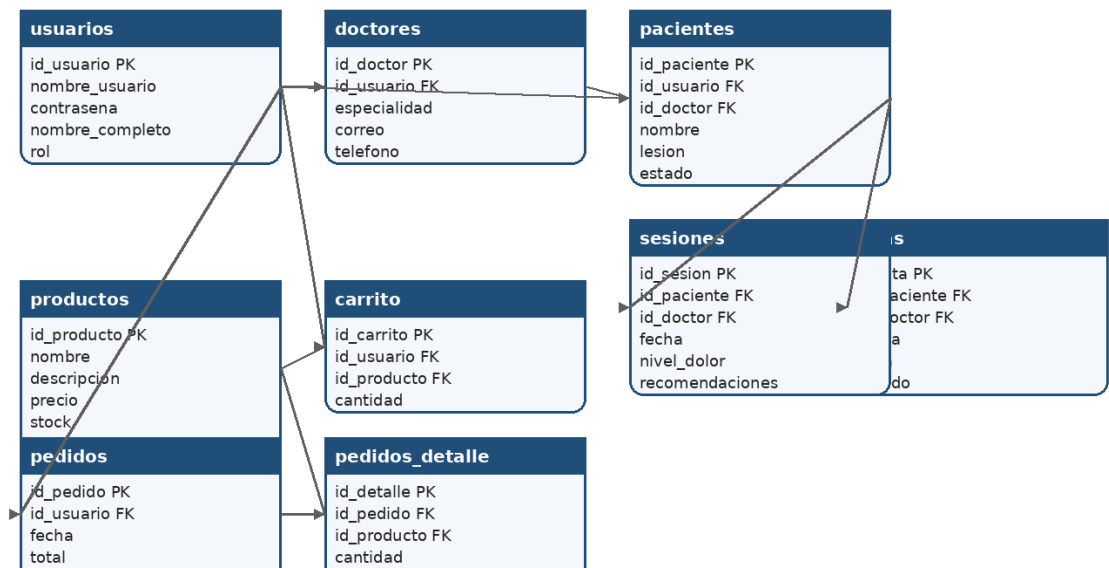


Figura 5. Diagrama relacional final generado a partir del SQL del proyecto.

Tabla	Propósito	Relaciones principales
usuarios	Controla credenciales, nombre completo y rol del usuario.	Se relaciona con doctores, pacientes, carrito y pedidos.
doctores	Almacena datos profesionales del doctor.	Depende de usuarios; atiende pacientes, citas y sesiones.
pacientes	Registra datos clínicos básicos y estado del tratamiento.	Puede tener usuario de portal; se relaciona con citas y sesiones.
citas	Programa atenciones por fecha y hora.	Pertenece a paciente y doctor.
sesiones	Documenta notas SOAP, dolor, ejercicios y recomendaciones.	Pertenece a paciente y doctor.
productos	Define artículos disponibles en tienda.	Se relaciona con carrito y pedidos_detalle.
carrito	Guarda productos seleccionados antes de compra.	Pertenece a usuario y producto.
pedidos	Registra compras realizadas por pacientes.	Pertenece a usuario y se desglosa en pedidos_detalle.
pedidos_detalle	Desglosa productos, cantidades y precios por pedido.	Relaciona pedidos con productos.

12. Implementación del sistema

La implementación final se entregó como un proyecto PHP/MySQL ejecutable en XAMPP. El código se organizó por módulos funcionales para facilitar su mantenimiento y lectura. La estructura del proyecto incluye vistas principales, configuración, recursos CSS/JS, imágenes de productos y script SQL de instalación.

Archivo / carpeta	Descripción
index.php	Pantalla de inicio de sesión.
login.php / logout.php	Procesamiento de autenticación y cierre de sesión.
admin_dashboard.php	Panel principal del administrador.
doctors.php	Alta y eliminación de doctores.
dashboard.php	Panel principal del doctor.
patients.php	Alta, listado y eliminación de pacientes.
appointments.php	Gestión de citas.
sessions.php	Registro de notas SOAP y consulta de historial clínico.
patient_portal.php	Portal de seguimiento para el paciente.
products.php	Gestión de productos de tienda para administrador.
store.php / cart.php / orders.php	Tienda, carrito y pedidos.
reports.php	Reportes clínicos y seguimiento.
assets/css/styles.css	Diseño visual, layout, formularios, tablas y responsividad.
assets/js/app.js	Comportamiento del lado cliente, formato de teléfono y ajustes de campos.
database/teraphybenefit.sql	Script de creación e inserción inicial de datos.

Tipo de archivo	Cantidad aproximada en el ZIP
.css	1
.js	1
.md	1
.php	17
.sql	1
.svg	6
.txt	2

13. Funcionalidades por rol

13.1 Administrador

El administrador general tiene acceso a un panel de administración donde puede revisar información global, dar de alta doctores, eliminar doctores y administrar los productos de la tienda. Esta separación evita que el doctor modifique la configuración general del sistema.

Función	Descripción
Gestión de doctores	Alta con usuario, contraseña segura, especialidad, correo y teléfono.
Gestión de productos	Alta de productos con nombre, imagen, precio, stock y descripción.
Consulta de pedidos	Visualización de pedidos realizados por pacientes.
Control de roles	Acceso restringido mediante funciones de helpers.php.

13.2 Doctor

El doctor opera los módulos clínicos. Puede registrar pacientes, crear usuario para portal, programar citas, capturar notas SOAP, revisar reportes y consultar la evolución de sus pacientes.

Función	Descripción
Alta de pacientes	Captura expediente, lesión, contacto, fecha de inicio, estado y nota inicial.
Citas	Programa fecha, hora, motivo y notas.
Notas SOAP	Documenta signos vitales, ejercicios, evolución, dolor y recomendaciones.
Reportes	Consulta resumen de evolución, sesiones y dolor promedio.

13.3 Paciente

El paciente accede a un portal simplificado para consultar su seguimiento clínico, revisar citas y visualizar recomendaciones. Además, cuenta con una tienda de productos terapéuticos y un carrito de compras.

Función	Descripción
Mi seguimiento	Consulta datos del expediente, estado del tratamiento y sesiones.
Historial de sesiones	Visualiza fechas, terapia realizada, dolor y recomendaciones.
Tienda	Consulta productos de apoyo para ejercicios.
Carrito y pedidos	Selecciona productos, confirma compra y revisa pedidos realizados.

14. Evidencia visual de implementación

Durante el proceso se realizaron capturas de configuración, base de datos y funcionamiento de la aplicación. Estas imágenes respaldan que el proyecto fue ejecutado localmente, que la base de datos fue importada y que los módulos principales fueron probados en navegador.

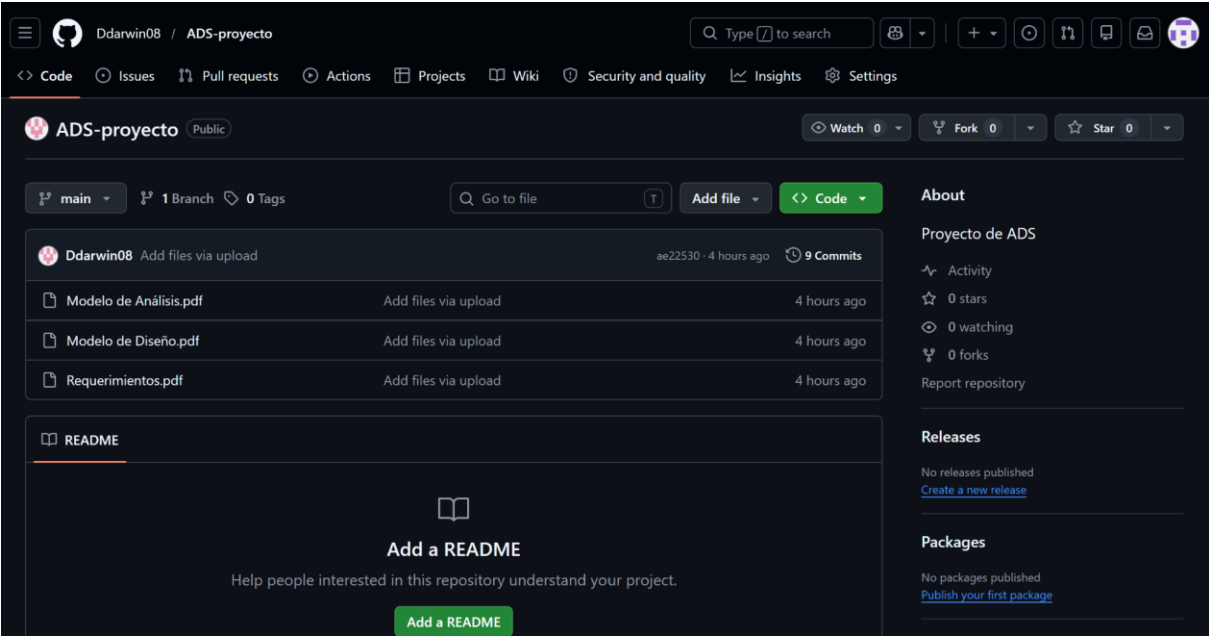


Figura 6. Repositorio de GitHub utilizado para organizar archivos del proyecto.

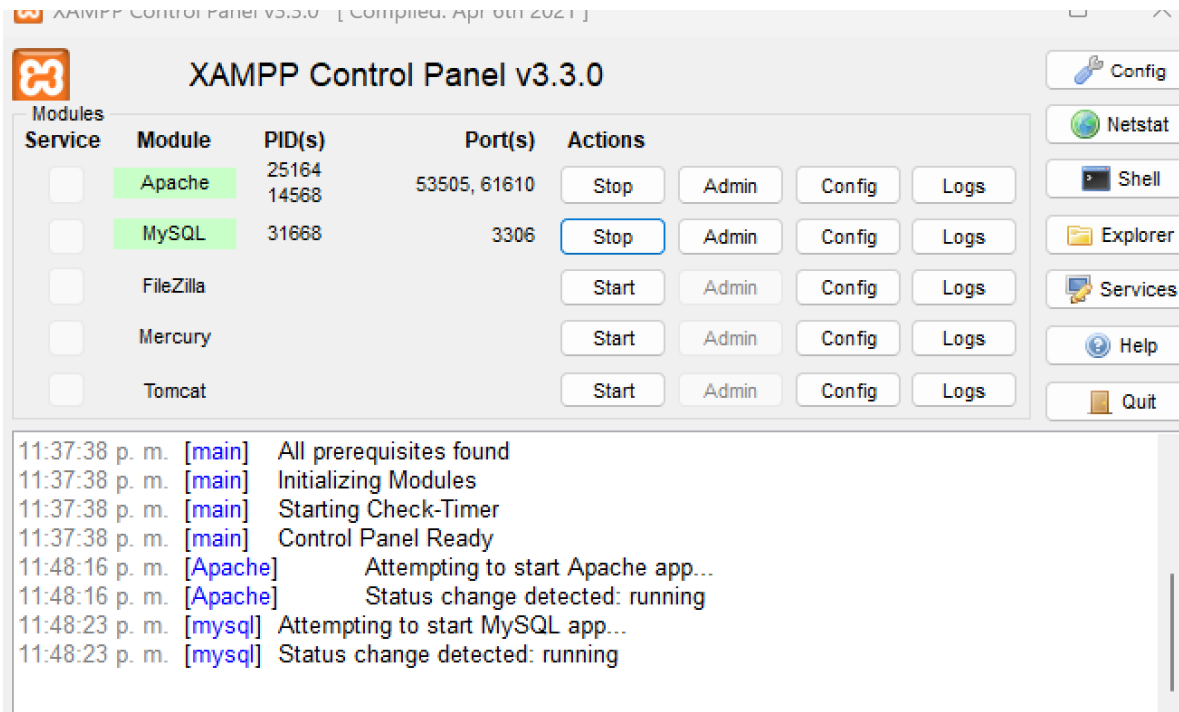


Figura 7. XAMPP con Apache y MySQL en ejecución.

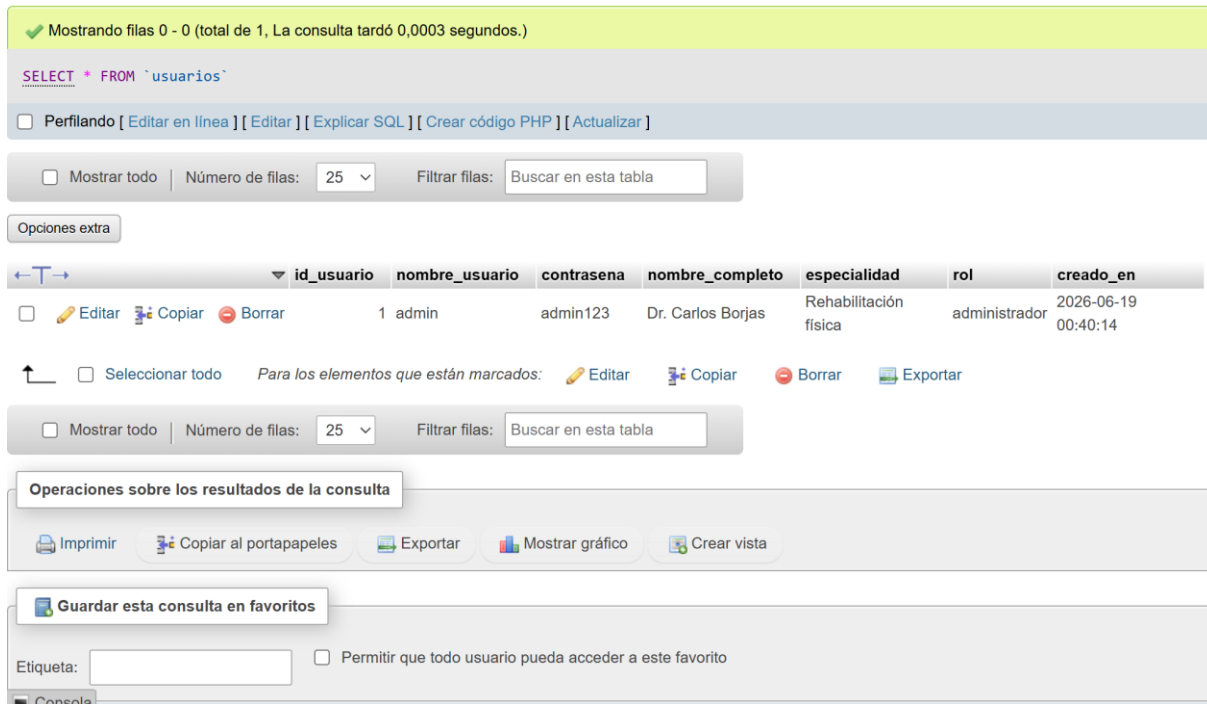


Figura 8. Consulta de tabla usuarios en phpMyAdmin con usuario administrador.

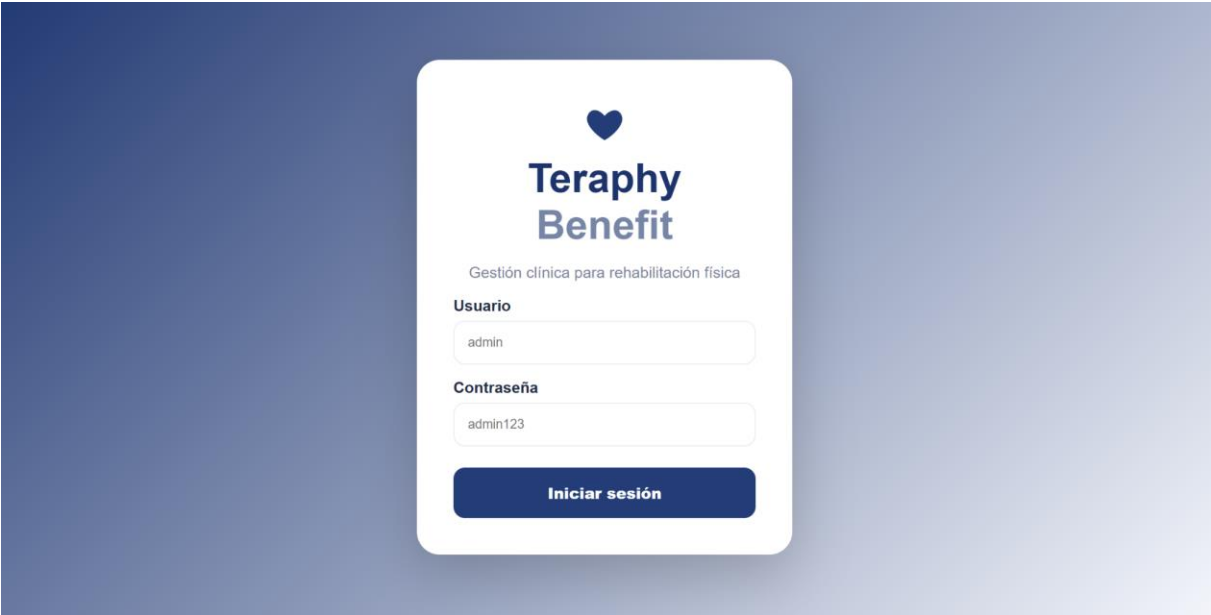


Figura 9. Pantalla de inicio de sesión de TeraphyBenefit.



Figura 10. Panel principal con métricas y próximas citas.

15. Prototipo visual y evolución hacia el sistema funcional

El prototipo visual elaborado por parte del equipo permitió validar la idea de una interfaz médica con navegación lateral, tarjetas de información y módulos organizados por secciones. Posteriormente, el prototipo fue transformado en una aplicación funcional conectada a MySQL, con persistencia real y control de roles.

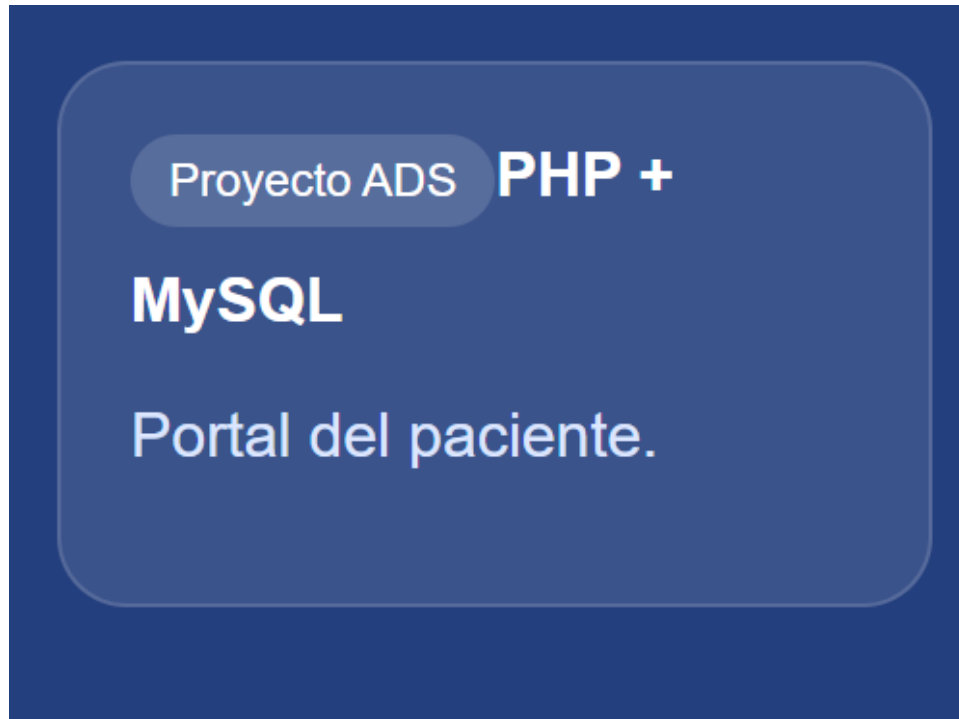


Figura 13. Propuesta inicial de tarjeta informativa del prototipo.



Figura 14. Referencia visual de marca usada durante el desarrollo.

El prototipo fue útil para decidir colores, jerarquía visual, navegación y distribución. La versión implementada mantuvo la identidad médica en tonos azules, la estructura de panel lateral y el enfoque en la experiencia del usuario.

16. Plan de pruebas y validación

Las pruebas se enfocaron en validar el flujo completo de la aplicación: autenticación, registro, persistencia, consulta, edición visual y separación de permisos por rol. La estrategia fue manual, ejecutando los casos directamente en el navegador con XAMPP y verificando resultados en phpMyAdmin.

ID	Módulo	Caso de prueba	Datos de entrada	Resultado esperado	Resultado
CP-01	Login	Inicio con admin.	admin / Admin123!	Acceso al panel admin.	Correcto
CP-02	Login	Inicio con doctor.	doctor1 / Doctor123!	Acceso al panel médico.	Correcto
CP-03	Login	Inicio con paciente.	javier / Javier123!	Acceso al portal del paciente.	Correcto
CP-04	Doctores	Alta de doctor.	Nombre, usuario, contraseña segura, especialidad.	Doctor registrado en MySQL.	Correcto
CP-05	Pacientes	Alta de paciente.	Nombre, edad, lesión, contacto, fecha.	Paciente aparece en lista.	Correcto
CP-06	Pacientes	Eliminar paciente.	Click en eliminar.	Registro y dependencias eliminadas.	Correcto
CP-07	Citas	Agendar cita.	Paciente, fecha, hora, motivo.	Cita registrada y visible.	Correcto
CP-08	SOAP	Registrar nota.	Fecha, terapia, dolor, ejercicios.	Sesión añadida al historial.	Correcto
CP-09	Tienda	Agregar producto.	Producto, precio, stock, imagen.	Producto visible en tienda.	Correcto
CP-10	Carrito	Compra de producto.	Agregar al carrito y comprar.	Pedido generado y carrito vacío.	Correcto
CP-11	Roles	Doctor intenta gestionar productos.	Acceso no permitido.	Función reservada al admin.	Correcto
CP-12	Validación visual	Formularios no desbordan panel.	Alta doctor/paciente/cita/SOAP/producto.	Campos contenidos.	Correcto

17. Manual de usuario resumido

17.1 Acceso al sistema

Para utilizar el sistema, el usuario debe iniciar XAMPP, activar Apache y MySQL, importar la base de datos y abrir el proyecto desde el navegador en la URL local. Después debe ingresar sus credenciales según el rol asignado.

Rol	Usuario de prueba	Contraseña	URL
Administrador	admin	Admin123!	http://localhost/teraphybenefit/
Doctor	doctor1	Doctor123!	http://localhost/teraphybenefit/
Paciente	javier	Javier123!	http://localhost/teraphybenefit/

17.2 Flujo del administrador

1. Iniciar sesión con usuario administrador.
2. Entrar al módulo Doctores para registrar o eliminar doctores.
3. Entrar al módulo Productos para registrar artículos de tienda.
4. Revisar pedidos generados por pacientes.

17.3 Flujo del doctor

5. Iniciar sesión con usuario doctor.
6. Registrar pacientes o crear usuario para portal del paciente.
7. Programar citas de seguimiento.
8. Capturar notas SOAP después de cada sesión.
9. Consultar reportes clínicos y evolución.

17.4 Flujo del paciente

10. Iniciar sesión con usuario paciente.
11. Consultar la sección Mi seguimiento.
12. Revisar historial de sesiones, dolor y recomendaciones.
13. Entrar a Tienda, agregar productos al carrito y realizar compra.
14. Consultar pedidos realizados.

18. Manual técnico resumido

Elemento técnico	Detalle
Servidor local	XAMPP con Apache y MySQL.
Lenguaje servidor	PHP.
Base de datos	MySQL/MariaDB, base teraphybenefit.
Frontend	HTML, CSS y JavaScript.
Conexión a BD	PDO en config/database.php.
Control de acceso	Sesiones PHP y funciones de rol en config/helpers.php.
Instalación	Copiar carpeta teraphybenefit a C:\xampp\htdocs e importar database/teraphybenefit.sql.

URL local de ejecución

<http://localhost/teraphybenefit/>

Si Apache utiliza un puerto personalizado, se debe anteponer el puerto correspondiente, por ejemplo:

<http://localhost:54780/teraphybenefit/>.

19. Seguridad y control de acceso

El sistema diferencia permisos mediante roles. Cada página valida que el usuario haya iniciado sesión y que tenga el rol necesario para acceder. El administrador no realiza funciones clínicas; el doctor no administra productos en la versión final; y el paciente solo consulta su información y utiliza tienda/carrito.

Recurso	Admin	Doctor	Paciente
Panel admin	Sí	No	No
Doctores	Sí	No	No
Pacientes	No	Sí	No
Citas	No	Sí	No
Notas SOAP	No	Sí	No
Reportes clínicos	No	Sí	No
Productos	Sí	No	No
Tienda	No	No	Sí
Carrito	No	No	Sí
Pedidos	Consulta general	Consulta general	Consulta propia

Como mejora futura se recomienda reemplazar el almacenamiento de contraseñas en texto plano por hashes con `password_hash()` y `password_verify()`, además de implementar tokens CSRF en formularios críticos.

20. Resultados obtenidos

La versión final logró integrar un sistema funcional con flujo completo de administración clínica. Se consiguió pasar de documentos de análisis y diseño a un producto ejecutable, con base de datos y módulos diferenciados. El resultado permite demostrar conocimientos de ingeniería de software, diseño de interfaces, programación web y persistencia de datos.

Aspecto evaluado	Resultado
Funcionalidad	Los módulos principales operan con base de datos real.
Diseño visual	La interfaz mantiene identidad médica, colores consistentes y navegación lateral.
Roles	El acceso se separa por administrador, doctor y paciente.
Persistencia	Los datos se almacenan en tablas relacionadas mediante MySQL.
Usabilidad	Los formularios fueron ajustados para evitar desbordamientos y mejorar legibilidad.
Extensibilidad	El diseño modular permite agregar funcionalidades futuras.

21. Conclusiones

El desarrollo de TeraphyBenefit permitió integrar de manera práctica las etapas del ciclo de vida del software. A partir de una necesidad real en el área de rehabilitación física, el equipo definió requerimientos, elaboró modelos de análisis y diseño, construyó una base de datos y finalmente implementó una aplicación web funcional. Este proceso evidenció la importancia de documentar antes de programar, ya que los artefactos sirvieron como guía para organizar módulos, actores, responsabilidades y restricciones del sistema.

Desde el punto de vista técnico, el proyecto fortaleció el uso de PHP y MySQL dentro de un entorno local con XAMPP. La implementación permitió comprender la relación entre formularios web, controladores, consultas SQL, validaciones y persistencia. También fue relevante la separación de roles, porque obligó a diseñar flujos específicos para administrador, doctor y paciente. Esta decisión mejora el control de acceso y hace que la aplicación sea más ordenada para cada tipo de usuario.

En cuanto a diseño de interfaz, el proyecto pasó por varias iteraciones. Primero se construyó un prototipo visual que ayudó a definir el estilo general; posteriormente se integró la funcionalidad real y se corrigieron detalles de distribución, formularios y visualización de tablas. Esta experiencia mostró que una interfaz no solo debe verse bien, sino también mantenerse estable cuando se capturan datos, se listan registros y se utilizan distintas resoluciones de pantalla.

La incorporación de la tienda y el portal del paciente amplió el valor del sistema. El paciente ya no queda únicamente como sujeto de datos, sino que puede consultar su seguimiento y adquirir productos de apoyo para ejercicios. Aunque esta funcionalidad puede seguir evolucionando, representa una base interesante para conectar el tratamiento clínico con recomendaciones prácticas de rehabilitación.

Finalmente, el trabajo colaborativo fue fundamental. Cada integrante aportó desde un área distinta: análisis, diseño, prototipo, implementación y pruebas. La distribución de responsabilidades permitió avanzar por etapas y corregir errores conforme se detectaban. TeraphyBenefit queda como una propuesta sólida y funcional, con posibilidades de crecimiento hacia notificaciones automáticas, reportes más avanzados, seguridad reforzada y despliegue en servidor real.

22. Trabajo futuro

- Implementar contraseñas cifradas con `password_hash()` y recuperación de contraseña por correo.
- Agregar edición de doctores, pacientes, citas, sesiones y productos.
- Integrar gráficas reales de evolución por paciente usando datos de sesiones.
- Agregar notificaciones de próximas citas por correo o WhatsApp.
- Exportar historial clínico en PDF para entrega al paciente o expediente.
- Implementar búsqueda avanzada y filtros por lesión, estado, doctor y fecha.
- Desplegar el sistema en un servidor web real con dominio y certificado HTTPS.
- Mejorar auditoría de cambios para saber quién modificó cada registro.

23. Anexos

23.1 Paquete entregable

El paquete entregable del código se proporcionó en formato ZIP con el nombre teraphybenefit. Para ejecutarlo, se debe copiar la carpeta en htdocs, iniciar XAMPP, importar el script SQL y abrir la URL local.

Elemento	Ruta / archivo
Código fuente	teraphybenefit/
Base de datos	teraphybenefit/database/teraphybenefit.sql
Configuración	teraphybenefit/config/database.php
Estilos	teraphybenefit/assets/css/styles.css
Scripts cliente	teraphybenefit/assets/js/app.js
Imágenes de productos	teraphybenefit/assets/img/

23.2 Evidencias adicionales

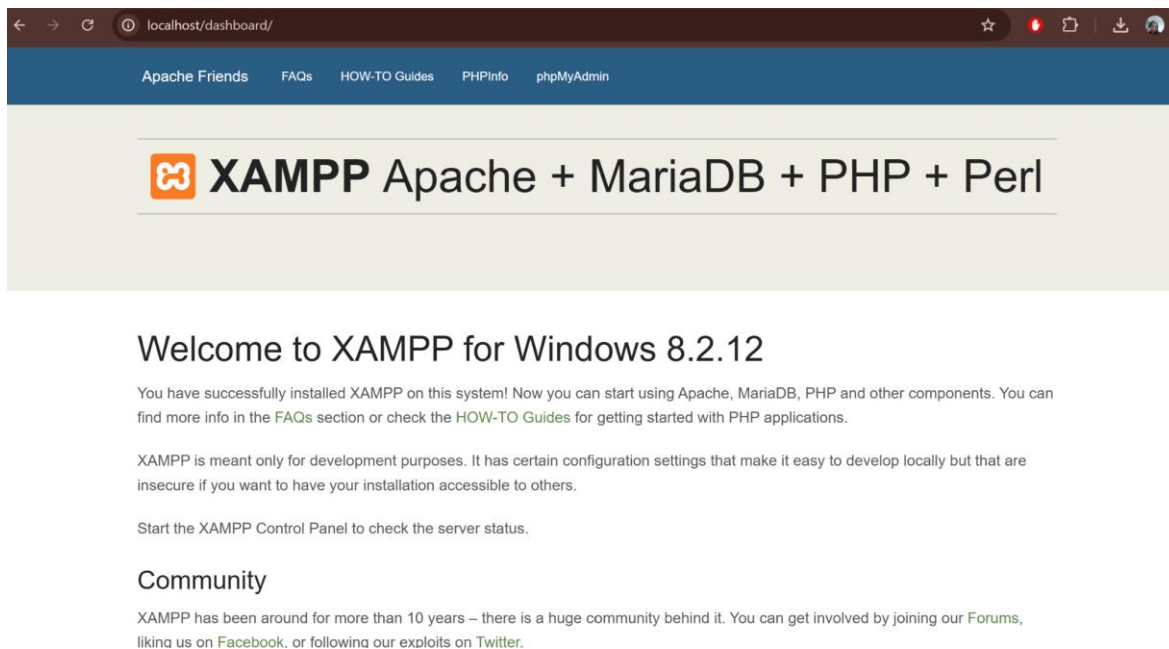


Figura 15. Página inicial de XAMPP funcionando en navegador.

✓ Importación ejecutada exitosamente, 17 consultas ejecutadas. (teraphybenefit.sql)

✓ MySQL ha devuelto un conjunto de valores vacío (es decir: cero columnas). (La consulta tardó 0,0005 segundos.)

```
CREATE DATABASE IF NOT EXISTS teraphybenefit CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
```

[Editar en línea] [Editar] [Crear código PHP]

⚠ Note: #1007 No puedo crear base de datos 'teraphybenefit'; la base de datos ya existe

✓ MySQL ha devuelto un conjunto de valores vacío (es decir: cero columnas). (La consulta tardó 0,0003 segundos.)

```
USE teraphybenefit;
```

[Editar en línea] [Editar] [Crear código PHP]

✓ MySQL ha devuelto un conjunto de valores vacío (es decir: cero columnas). (La consulta tardó 0,0004 segundos.)

```
DROP TABLE IF EXISTS sesiones;
```

[Editar en línea] [Editar] [Crear código PHP]

⚠ Note: #1051 Tabla 'teraphybenefit.sesiones' desconocida

Figura 16. Importación SQL ejecutada correctamente en phpMyAdmin.

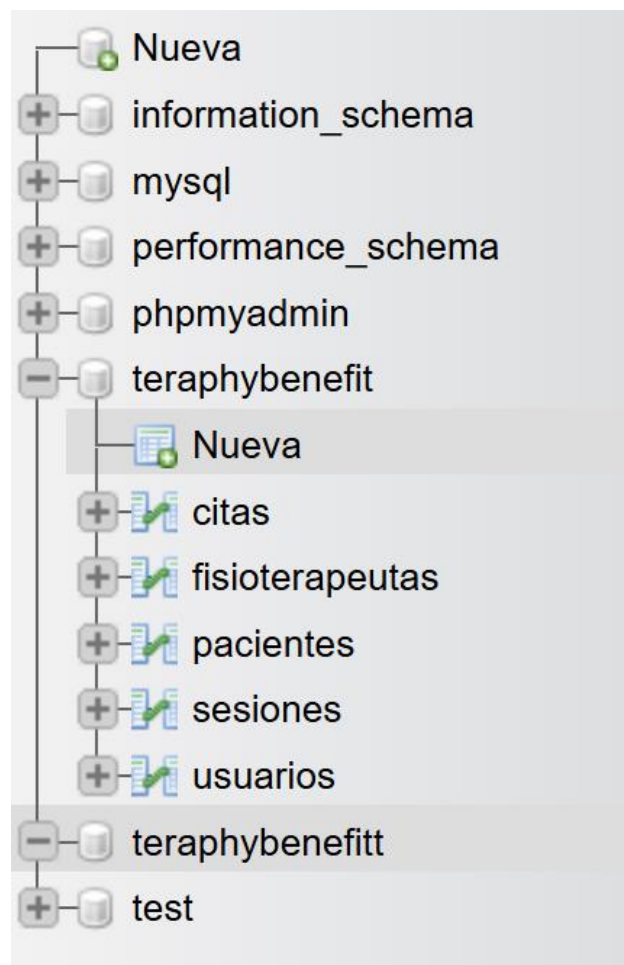


Figura 17. Base de datos teraphybenefit con tablas principales.